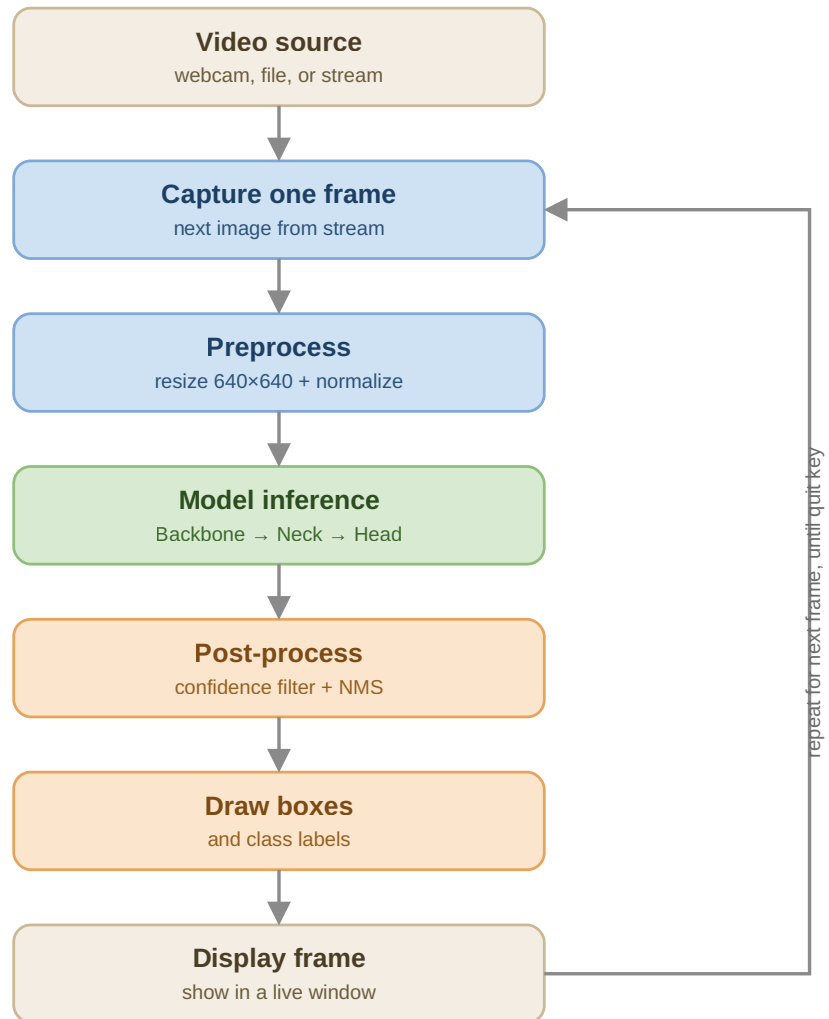


YOLOv8 — Real-time Detection Notes

Object Detection · Deep Learning Roadmap

Video source → capture → preprocess → inference → post-process → draw → display



The real-time loop — every stage repeats once per frame until the quit key is pressed.

Step 1 — Video source

- The live feed can come from a webcam, a saved video file, or an RTSP/IP camera stream
- All 3 sources are treated the same way once connected — as a continuous stream you pull frames from, one at a time

Note: Always confirm the source connected successfully before continuing — a wrong camera index or file path fails silently otherwise.

Step 2 — Capture one frame

- A video is really just a rapid sequence of individual images (frames), shown one after another
- Since our model only processes one image at a time, we pull exactly one frame from the stream at a time
- This happens inside a continuous loop — every loop iteration grabs the next fresh frame

Step 3 — Preprocess

- The captured frame is resized to 640×640, matching what the model was trained on
- Pixel values are normalized into a 0 to 1 range, and colors are converted into the order the model expects

Note: This step happens automatically inside the model call — no manual preprocessing needed in the actual pipeline.

Step 4 — Model inference

- The preprocessed frame passes through the full trained architecture: Backbone → Neck → Head
- This is a pure forward pass only — no gradients, no backward pass, since we're not training anymore
- Much faster than training, since there's far less computation happening per frame

Note: The model should be loaded once, before the frame loop starts. Reloading it every frame would badly hurt real-time speed.

Step 5 — Post-process (confidence filter + NMS)

- Raw predictions are messy — the same real object often gets multiple overlapping boxes
- Confidence filtering removes any box the model isn't confident enough about
- NMS (Non-Max Suppression) then removes duplicate overlapping boxes, keeping only the highest-confidence one per object

Note: NMS decides overlap using IoU (Intersection over Union) — boxes overlapping past a threshold are treated as duplicates of the same object.

Step 6 — Draw boxes and labels

- For each cleaned-up detection, a rectangle is drawn around the object on the frame
- The class name and confidence score are written just above the box
- This is purely a visualization step — it doesn't change any of the model's actual predictions

Step 7 — Display frame

- The annotated frame is shown in a live window, which keeps updating as new frames are processed
- A short wait after each frame lets the window refresh and checks for a key press to stop the loop

Note: Live window display doesn't work directly inside Google Colab — it needs a local Python environment, or a snapshot-style workaround inside notebooks.

The full loop, conceptually

- Open the video source once, and load the trained model once, before the loop starts
- Inside the loop: capture a frame, run it through the model, clean up the predictions, draw the results, then display the frame
- Repeat this continuously until a quit key is pressed, then release the video source and close the window

ARCHITECTURE
Done

TRAINING PIPELINE
Done

REAL-TIME
DETECTION
Done — FPS
optimization
pending

NEXT TOPIC
RNN